

Apprentissage par renforcement en action continue : PPO from scratch sur BipedalWalker-v3

Giuliano ALDARWISH, Mayy MILED

16 février 2026

Résumé

Ce rapport présente une implémentation *from scratch* de Proximal Policy Optimization (PPO) appliquée à l'environnement **BipedalWalker-v3** (contrôle continu). Nous détaillons (i) le cadre expérimental et la mesure de performance, (ii) la modélisation acteur-critique avec politique gaussienne *squashée* par $\tanh$, (iii) la collecte on-policy et l'estimation d'avantages via $GAE(\lambda)$, (iv) l'optimisation PPO avec objectif *clipped* et diagnostics de stabilité (KL, clip fraction, explained variance), (v) la normalisation des observations `RunningMeanStd` et son couplage indispensable à l'évaluation. Nous reportons enfin les résultats d'évaluation et discutons la robustesse (sensibilité à la seed, variance inter-épisodes) ainsi qu'une tentative d'implémentation de SAC ayant convergé vers un comportement dégénéré.

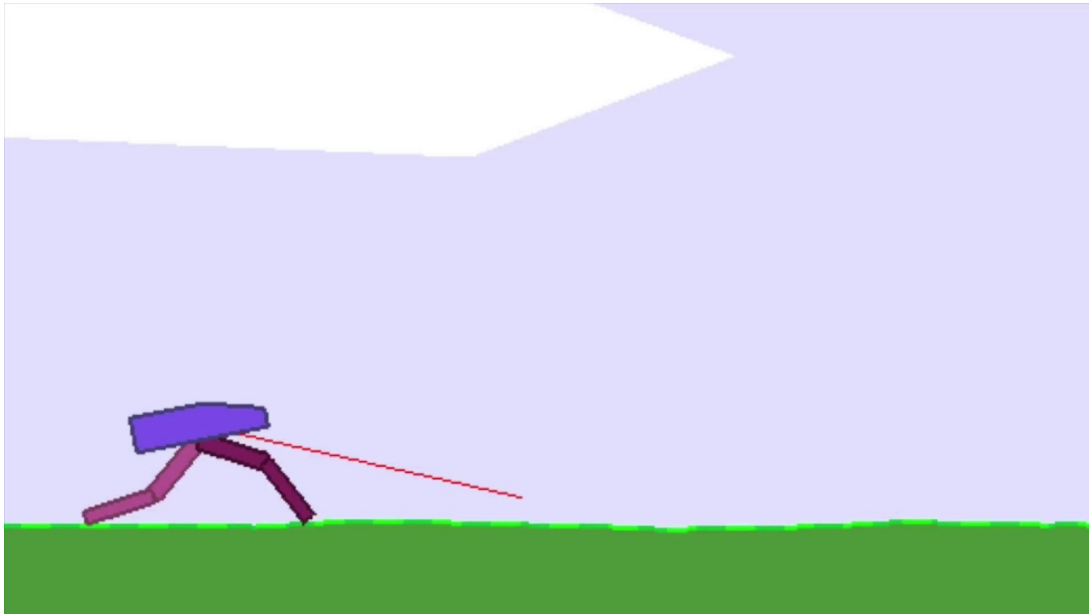


FIGURE 1 – Bipedal Walker by OpenAI Gym

Table des matières

1	Introduction	3
2	Environnement : BipedalWalker-v3	3
2.1	Description	3
2.2	Récompense et métrique de performance	3
3	Choix de l’algorithme : Proximal Policy Optimization (PPO)	3
3.1	Motivation (pourquoi PPO ici)	3
3.2	Rappel théorique : objectif PPO <i>clipped</i>	4
4	Implémentation (architecture et pipeline)	4
4.1	Organisation du code	4
4.2	Politique : gaussienne <i>squashée</i> par tanh	4
4.3	Critique : approximation de la valeur	5
4.4	Collecte on-policy : RolloutBuffer	5
4.5	Estimation des avantages : GAE(λ)	5
4.6	Update PPO : minibatches, epochs et diagnostics	5
4.7	Normalisation des observations : RunningMeanStd	6
4.8	Évaluation périodique et sauvegarde des meilleurs modèles	6
4.9	Instrumentation : TensorBoard, vidéos, export des courbes	6
5	Protocole expérimental	6
5.1	Hyperparamètres principaux	6
5.2	Variantes étudiées (robustesse et exploration)	7
6	Résultats	7
6.1	Évaluation finale (checkpoint <code>_best.pt</code>)	7
6.2	Comparaison des performances pendant l’entraînement	8
6.3	Zoom sur la stabilité PPO (diagnostics sur un run)	10
6.4	Analyse : performance vs robustesse	11
6.5	Analyse qualitative : vers une locomotion réellement bipède	12
7	Tentative SAC : analyse d’un échec et enseignements	12
8	Discussion : limites et pistes d’amélioration	12
8.1	Sensibilité aux seeds et variance inter-épisodes	12
8.2	Exploration via entropie	13
8.3	Pistes techniques (sans changer l’algorithme de base)	13
9	Conclusion	13

1 Introduction

L'apprentissage par renforcement en action continue pose un compromis central : obtenir une politique performante tout en gardant une optimisation stable malgré la variance des trajectoires et la non-stationnarité induite par l'amélioration de la politique. Ce projet vise à entraîner un agent sur **BipedalWalker-v3**, un benchmark de locomotion où l'agent contrôle un bipède via des actions continues (couples articulaires) afin d'avancer sans chuter.

Le choix principal du projet est de partir de PPO, algorithme acteur-critique *on-policy* réputé robuste, plutôt que de méthodes *off-policy* plus sensibles aux détails d'implémentation (SAC/TD3) dans ce contexte. L'objectif est double :

- **Performance** : obtenir un retour moyen élevé sur plusieurs épisodes d'évaluation (avec politique déterministe à l'évaluation).
- **Qualité d'ingénierie** : fournir une implémentation claire, modulaire, instrumentée (TensorBoard, checkpoints, vidéos), et un protocole expérimental reproductible.

2 Environnement : BipedalWalker-v3

2.1 Description

BipedalWalker-v3 est un environnement Gymnasium de locomotion où l'agent observe un état continu $s_t \in \mathbb{R}^d$ (angles, vitesses, informations de contact, etc.) et produit une action continue $a_t \in \mathbb{R}^m$ correspondant aux commandes des moteurs. Les actions sont bornées, typiquement dans $[-1, 1]$ composante par composante, d'où l'usage courant d'une politique dont la sortie est *squashée* via $\tanh$.

2.2 Récompense et métrique de performance

À chaque pas, l'agent reçoit une récompense r_t (façonnée) qui encourage la progression vers l'avant et pénalise des comportements instables (chutes, consommation excessive, etc.). La performance est mesurée par le **retour cumulé** :

$$R = \sum_{t=0}^{T-1} r_t,$$

où T est la longueur de l'épisode (terminaison ou troncature). Dans tout le rapport, nous reportons :

- le **retour moyen** sur N épisodes d'évaluation,
- l'**écart-type**, ainsi que **min/max**,
- la **longueur moyenne** des épisodes.

Ce choix est cohérent avec l'objectif final : un agent dont les performances sont non seulement élevées en moyenne, mais également *stables* (faible variance).

3 Choix de l'algorithme : Proximal Policy Optimization (PPO)

3.1 Motivation (pourquoi PPO ici)

PPO est un algorithme acteur-critique *on-policy* qui met l'accent sur la stabilité des mises à jour. Dans le cadre de **BipedalWalker-v3** :

- l'action est **continue** : PPO s'adapte naturellement via une politique gaussienne,
- la dynamique est **complexe** et l'apprentissage peut diverger : l'objectif *clipped* réduit les mises à jour trop agressives,

- la mise en place est **diagnosticable** : les métriques (KL approx, clipfrac, explained variance) permettent de relier la stabilité de l’optimisation à la performance.

Dans une logique de projet, PPO constitue un excellent *socle* : suffisamment simple pour une implémentation propre from scratch, tout en restant performant sur ce type de tâche.

3.2 Rappel théorique : objectif PPO *clipped*

On note $\pi_\theta(a|s)$ la politique actuelle et $\pi_{\theta_{\text{old}}}$ la politique qui a généré les trajectoires. Le ratio d’importance est :

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}.$$

Avec l’avantage estimé $\hat{A}_t$, l’objectif PPO *clipped* est :

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

La loss totale combine politique, valeur et entropie :

$$\mathcal{L}(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_v \mathbb{E}_t [(V_\phi(s_t) - \hat{R}_t)^2] - c_e \mathbb{E}_t [\mathcal{H}(\pi_\theta(\cdot|s_t))],$$

où $\hat{R}_t$ est une cible de retour (via GAE/returns), c_v le poids de la loss valeur, et c_e le coefficient d’entropie (exploration).

4 Implémentation (architecture et pipeline)

4.1 Organisation du code

L’implémentation est structurée en modules, de façon à séparer clairement les responsabilités :

- `train_core.py` : boucle d’entraînement, instrumentation (TensorBoard), checkpoints, vidéos, export des courbes.
- `model.py` : architecture ActorCritic, et fonctions de sampling (action + log-prob).
- `buffer.py` : RolloutBuffer et calcul GAE.
- `update.py` : étape `ppo_update` (minibatches, epochs, losses, diagnostics).
- `normalize.py` : RunningMeanStd et normalisation des observations.
- `scripts/eval.py` : évaluation sur épisodes, vidéo optionnelle, chargement checkpoint + stats de normalisation.

Cette décomposition rend le pipeline lisible, testable, et facilite l’analyse ablation (hyperparamètres, seeds).

4.2 Politique : gaussienne *squashée* par tanh

La politique produit des paramètres $(\mu_\theta(s), \sigma_\theta(s))$ et définit une distribution gaussienne :

$$u \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)), \quad a = \tanh(u).$$

Le *squashing* est crucial pour respecter les bornes d’action. À l’entraînement, l’action est **stochastique** (échantillonnage), tandis qu’à l’évaluation nous considérons une politique **déterministe** $a = \tanh(\mu_\theta(s))$, afin de mesurer une performance reproductible.

4.3 Critique : approximation de la valeur

Le critique approxime $V_\phi(s) = \mathbb{E}[R|s]$. Dans l'implémentation, le réseau **ActorCritic** fournit :

- (μ, σ) pour la politique,
- v pour la valeur $V(s)$.

Cette structure est standard en acteur-critique, et permet d'utiliser des avantages basés sur la valeur (GAE).

4.4 Collecte on-policy : RolloutBuffer

À chaque **update** PPO, le script collecte un rollout de longueur fixe (`rollout_len = 2048`). Le buffer stocke :

$$(s_t, a_t, \log \pi_{\theta_{\text{old}}}(a_t|s_t), r_t, d_t, V_\phi(s_t)).$$

Le choix de 2048 est classique pour PPO : suffisamment long pour estimer des avantages informatifs, tout en gardant un coût mémoire/temps raisonnable.

4.5 Estimation des avantages : GAE(λ)

Nous utilisons Generalized Advantage Estimation avec $(\gamma, \lambda) = (0.99, 0.95)$. En notant

$$\delta_t = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t),$$

l'avantage est estimé par :

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}.$$

Les **returns** utilisés pour la loss valeur sont ensuite :

$$\hat{R}_t = \hat{A}_t + V(s_t).$$

Ce choix ($\lambda = 0.95$) est un compromis standard biais/variance bien adapté aux tâches de locomotion.

4.6 Update PPO : minibatches, epochs et diagnostics

Après collecte, nous effectuons une mise à jour PPO via :

- **10 epochs** sur les données on-policy,
- **minibatch size** de 64,
- **gradient clipping** : `max_grad_norm=0.5`,
- **learning rate schedule** décroissant linéaire (facilite la convergence en fin d'apprentissage),
- **value clipping** optionnel (activé ici) pour stabiliser la tête valeur.

La stabilité de PPO est monitorée via :

- **approx_kl** : divergence KL approximative entre politiques,
- **clipfrac** : fraction des ratios r_t ayant été *clippés*,
- **explained_var** : qualité d'ajustement de la valeur (proxy du signal de critic).

Ces signaux sont essentiels pour relier la dynamique d'optimisation (sur/sous-apprentissage) aux variations de performance.

4.7 Normalisation des observations : `RunningMeanStd`

Un point technique déterminant du projet est la normalisation des observations. Nous maintenons une estimation en ligne :

$$\mu \leftarrow \text{mean}(s), \quad \sigma^2 \leftarrow \text{var}(s),$$

et appliquons :

$$\tilde{s} = \frac{s - \mu}{\sqrt{\sigma^2 + \epsilon}}.$$

Point critique : la normalisation doit être *cohérente* entre train et évaluation. Dans ce projet, les statistiques `mean/var/count` sont sauvegardées dans le checkpoint et rechargées dans `eval.py`. Sans cela, la distribution d’entrée vue par le réseau à l’évaluation change, ce qui dégrade fortement la performance (erreur fréquente en pratique).

4.8 Évaluation périodique et sauvegarde des meilleurs modèles

L’entraînement effectue une évaluation régulière toutes les `evaluation_interval` updates, sur 5 épisodes, avec une politique déterministe. Le meilleur modèle est sauvegardé (`_best.pt`) si le retour moyen d’évaluation s’améliore. Ce mécanisme :

- fournit un critère simple de sélection de modèle,
- protège contre des régressions temporaires,
- facilite la reproductibilité des résultats reportés.

4.9 Instrumentation : TensorBoard, vidéos, export des courbes

L’entraînement logge les métriques clés dans TensorBoard (retour épisode, losses, KL, etc.). De plus :

- des vidéos sont enregistrées périodiquement via `RecordVideo`,
- les courbes TensorBoard sont exportées automatiquement en PNG via `EventAccumulator` (`save_learning_curves`).

Cela permet une intégration directe dans le rapport, et une analyse qualitative du comportement appris.

5 Protocole expérimental

5.1 Hyperparamètres principaux

Les expériences principales utilisent les paramètres suivants (config de base PPO) :

Paramètre	Valeur
Environnement	BipedalWalker-v3
Budget d'interactions	2 000 000 steps
Rollout length	2048
γ / λ	0.99 / 0.95
Clip ϵ	0.2
Learning rate	3×10^{-4} (décroissance linéaire)
Epochs / Minibatch	10 / 64
Value loss coef c_v	0.5
Entropy coef c_e	0.0 (puis variantes à 0.005)
Max grad norm	0.5
Value clipping	activé
Évaluation	toutes les 25 updates, 5 épisodes
Checkpoint	toutes les 25 updates + meilleur modèle

TABLE 1 – Hyperparamètres PPO (configuration de base).

5.2 Variantes étudiées (robustesse et exploration)

Au-delà de la configuration de base, trois configurations comparatives structurent l'analyse :

- **Run A (baseline)** : seed=0, steps=2M, $c_e = 0.0$.
- **Run B (budget + exploration)** : seed=0, steps=4M, $c_e = 0.005$.
- **Run C (robustesse / seed)** : seed=42, steps=2M, $c_e = 0.0$.

L'objectif est de séparer (i) l'effet du *budget* et de l'*exploration* (A vs B) et (ii) l'effet de la *seed* à budget constant (A vs C).

6 Résultats

6.1 Évaluation finale (checkpoint `_best.pt`)

Nous présentons ici une évaluation sur $N = 10$ épisodes, en utilisant la politique déterministe ($a = \tanh(\mu)$) et la normalisation des observations rechargée depuis le checkpoint.

Résultats obtenus (Run A : seed=0, budget 2M, $c_e = 0.0$) :

- **Retour moyen** : 284.5 ± 44.4 ,
- **Min/Max** : 151.4 / 301.3,
- **Longueur moyenne** : 1019.9.

Résultats obtenus (Run B : seed=0, budget 4M, $c_e = 0.005$) :

- **Retour moyen** : 314.4 ± 0.7 ,
- **Min/Max** : 312.8 / 315.1,
- **Longueur moyenne** : 706.9.

Résultats obtenus (Run C : seed=42, budget 2M, $c_e = 0.0$) :

- **Retour moyen** : 313.0 ± 2.1 ,
- **Min/Max** : 308.6 / 316.1,
- **Longueur moyenne** : 819.8.

Configuration	Mean return	Std	Min/Max	Mean length
Run A : seed=0, steps=2M, $c_e = 0.0$	284.5	44.4	151.4 / 301.3	1019.9
Run B : seed=0, steps=4M, $c_e = 0.005$	314.4	0.7	312.8 / 315.1	706.9
Run C : seed=42, steps=2M, $c_e = 0.0$	313.0	2.1	308.6 / 316.1	819.8

TABLE 2 – Synthèse des performances en évaluation (10 épisodes).

6.2 Comparaison des performances pendant l’entraînement

La comparaison la plus informative est la courbe de retour moyen en évaluation au fil des updates (`eval/avg_return`), car elle mesure la performance de manière périodique avec une politique déterministe, indépendamment du bruit d’échantillonnage intra-épisode.

Les figures 2 et 3 ci-dessous présentent les retour moyen en évaluation des run B et run C comparé au run A.

On observe d’abord une phase de progression rapide, puis un plateau.

Sur la figure 3, nous observons en rose que le Run B (steps =4M, $c_e = 0.005$, seed=0) a un return qui oscille bien plus que celui du run A au début, notamment grâce à plus d’exploration. Les retours augmentent ensuite nettement au fil des mises à jour et convergent vers une valeur légèrement supérieure à celle du run A. L’exploration n’a pas non plus été drastiquement augmentée car l’agent était très sensible à tout changement et trop d’exploration réduisait nettement son apprentissage. Ainsi, pour le run C, nous observons un plateau plus stable et un niveau final élevé.

La figure 2 compare le run C (seed=42, 2M) au run A de base. Nous observons à budget constant et sans entropie, une trajectoire d’apprentissage plus favorable qui peut aussi conduire à une politique robuste. Ce simple changement de seed montre qu’à l’update 100 par exemple le retour est presque doublé comparé au run A (100 contre 190). À l’inverse, le Run A (seed=0, 2M) est nettement moins stable en évaluation : il obtient de très bons épisodes mais présente une variance plus importante, ce qui motive l’analyse multi-seeds et/ou un budget plus grand.

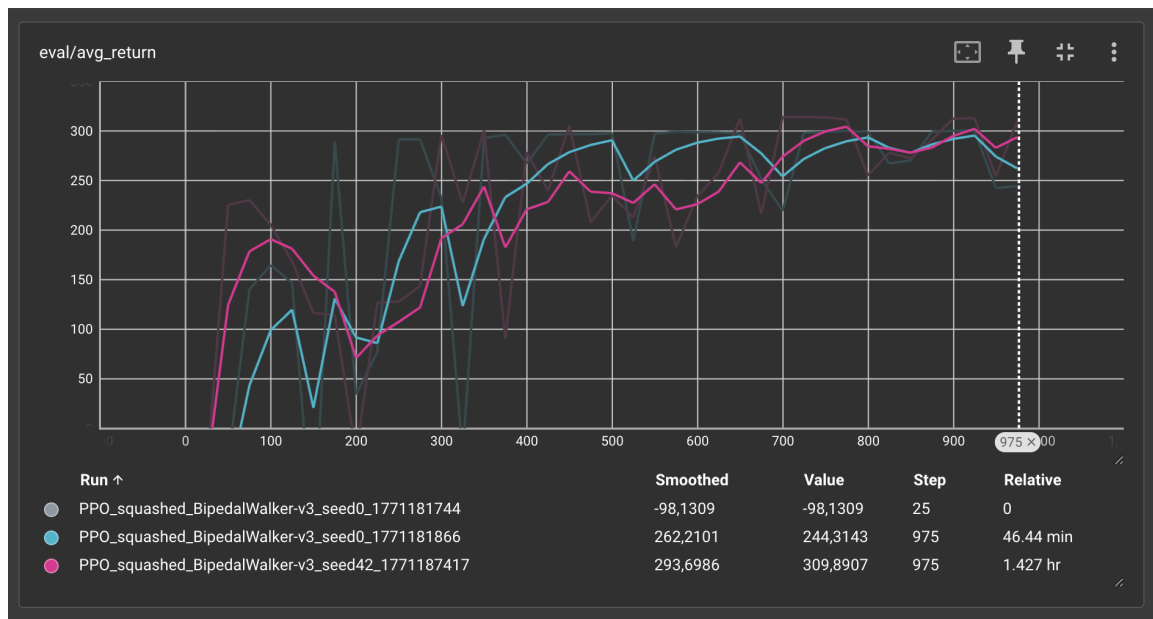


FIGURE 2 – Retour moyen en évaluation (eval/avg_return) pour le run C avec seed= 42 (en rose) comparé au run A (en bleu)

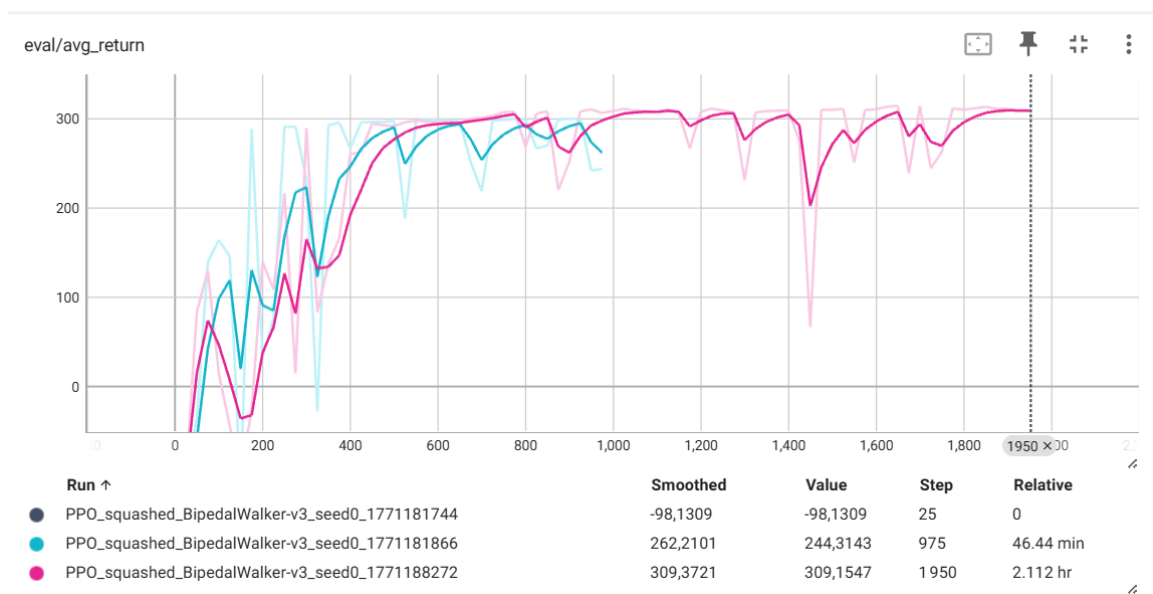


FIGURE 3 – Retour moyen en évaluation (eval/avg_return) pour le run B (seed = 0)

6.3 Zoom sur la stabilité PPO (diagnostics sur un run)

Au-delà de la performance, il est utile de vérifier que l’optimisation reste dans un régime contrôlé. Nous illustrons cela sur le Run B, qui sert ici de run “référence” (car c’est celui qui est le plus stable en évaluation).

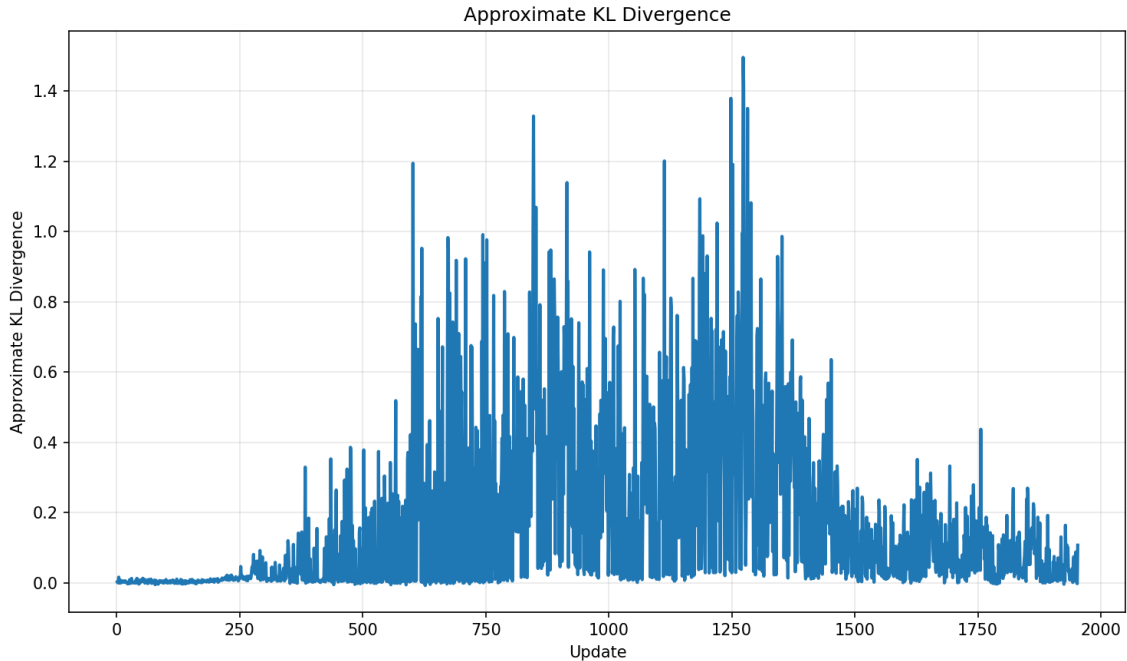


FIGURE 4 – Divergence KL approximative (diagnostics/approx_kl) — Run B.

La Fig. 4 montre une dynamique en **trois phases** plutôt qu’un niveau de KL constant. Au début de l’entraînement (environ jusqu’à 200–300 updates), la divergence KL est **très proche de zéro**, ce qui correspond à des mises à jour faibles : la politique change peu d’une itération à l’autre. Ensuite, entre ~ 300 et ~ 1400 updates, on observe une **forte augmentation** de la KL avec une grande variabilité : la courbe présente de nombreux pics, souvent entre 0.2 et 0.8, et quelques pointes atteignant **au-delà de 1** (jusqu’à ~ 1.5). Cette zone correspond à une période où les mises à jour PPO deviennent beaucoup plus **agressives**, ce qui est cohérent avec une phase d’amélioration rapide de la politique (mais aussi un risque accru d’instabilité si ces pics devenaient systématiques).

Enfin, après ~ 1400 updates, la KL **redescend nettement** et se stabilise dans une plage plus basse (typiquement < 0.2) avec quelques pics isolés. Ce retour à un régime plus conservateur est cohérent avec une phase de **consolidation** en fin d’apprentissage : la politique continue de s’ajuster, mais avec des pas plus petits, ce qui accompagne généralement un plateau de performance plus stable en évaluation.

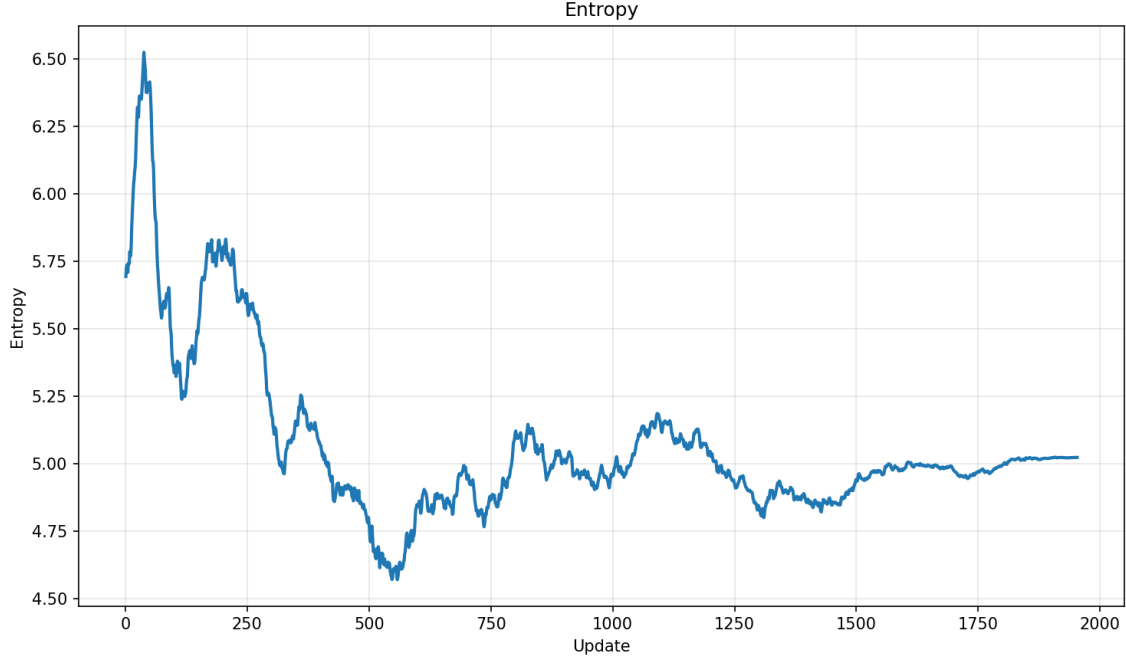


FIGURE 5 – Entropie de la politique (loss/entropy) — Run B ($c_e = 0.005$).

La Fig. 5 met en évidence une évolution **non monotone** de l’entropie. Au tout début, l’entropie est élevée (avec un **pic** au-dessus de 6), ce qui traduit une politique très stochastique : l’agent explore fortement et les actions sont variées. Ensuite, sur les premières centaines d’updates, l’entropie **baisse rapidement** et atteint un minimum autour de 4.6–4.8 (vers ~ 500 updates), signe que la politique devient nettement plus **déterministe** en se spécialisant sur une stratégie qui fonctionne.

Après cette phase, l’entropie **remonte légèrement** puis se **stabilise** autour de ~ 5.0 en fin d’entraînement, avec de petites oscillations. Ce comportement est cohérent avec l’ajout d’un coefficient d’entropie non nul ($c_e = 0.005$) : la politique se spécialise, mais l’optimisation conserve une **diversité d’actions** résiduelle au lieu de converger vers une politique quasi déterministe. Dans notre cas, cette exploration résiduelle est compatible avec les performances d’évaluation très stables du Run B, car elle peut réduire le risque de sur-spécialisation à des trajectoires étroites et améliorer la robustesse face à des états rarement rencontrés.

6.4 Analyse : performance vs robustesse

Les trois runs donnent une image plus complète que la seule baseline.

Effet de la seed (Run A vs Run C, budget constant 2M, $c_e = 0.0$). Le passage de seed=0 à seed=42 modifie fortement le résultat : Run C atteint un retour moyen ~ 313 avec une faible variance, alors que Run A présente une variance élevée et des épisodes nettement plus faibles (min=151.4). Cela illustre un point important sur ce type de benchmark : une seule seed peut donner une vision biaisée de la qualité réelle de l’approche. Dans notre cas, Run C indique que la configuration “2M, $c_e = 0$ ” *peut* produire une politique robuste, mais que ce n’est pas garanti (Run A).

Effet budget + exploration (Run A vs Run B, seed constante). À seed=0, augmenter le budget à 4M et maintenir une entropie non nulle ($c_e = 0.005$) produit une politique très stable (écart-type 0.7 sur 10 épisodes) et un retour moyen plus élevé. Empiriquement, cela va dans le sens d’un entraînement qui “consolide” la stratégie : l’agent explore encore suffisamment pour

éviter certains minima locaux, puis se spécialise progressivement, ce qui donne une performance régulière.

6.5 Analyse qualitative : vers une locomotion réellement bipède

Dans ce projet, le score n’était pas le seul objectif : on voulait aussi que l’agent apprenne une marche **réellement bipède**, c’est-à-dire qu’il utilise activement ses deux jambes avec une alternance d’appuis, plutôt qu’une stratégie ‘astucieuse’ qui avance sans marcher correctement.



Sur la configuration `PPO_squashed_BipedalWalker-v3_seed0_1771181866` (Run A : budget 2M, $c_e = 0.0$), l’agent obtient de très bons retours en évaluation, mais on observe un comportement asymétrique : il avance surtout en s’appuyant sur une seule jambe, pendant que l’autre est peu utilisée et finit souvent par être *traînée*. Autrement dit, il a trouvé une stratégie qui ‘marche’ du point de vue de la récompense, mais qui n’est pas une marche bipède stable au sens intuitif.

Une seconde configuration expérimentale, `PPO_squashed_BipedalWalker-v3_seed0_1771188272` (Run B), a été entraînée avec un budget d’interactions plus grand (4M) et une exploration légèrement renforcée via un coefficient d’entropie non nul ($c_e = 0.005$). Dans ce cas, le comportement observé devient plus symétrique : l’agent mobilise davantage les deux jambes et l’alternance des appuis est plus visible, ce qui se rapproche mieux de la locomotion attendue.

Enfin, la configuration `PPO_squashed_BipedalWalker-v3_seed42_1771187417` (Run C) montre qu’à budget constant (2M, $c_e = 0.0$), une politique robuste et performante peut également émerger selon la seed. Cette observation renforce l’idée que la ‘qualité locomotrice’ (symétrie, régularité) et la robustesse doivent être évaluées sur plusieurs runs, et pas uniquement sur un résultat isolé.

7 Tentative SAC : analyse d’un échec et enseignements

En parallèle, une tentative d’implémentation de Soft Actor-Critic (SAC) a été réalisée, mais l’agent a convergé vers un comportement dégénéré (immobilisation prolongée en posture de ‘grand écart’). Nous en avons déduit que la politique a trouvé un optimum local exploitable par la récompense (ou insuffisamment pénalisé). Nous avons eu du mal à calibrer les hyperparamètres d’entropie et l’échelle de récompense.

Dans le cadre du projet, ce résultat renforce la pertinence de PPO comme point de départ : le pipeline on-policy, les diagnostics, et la simplicité relative de mise au point rendent l’itération plus fiable. Une suite logique serait de reprendre SAC avec un protocole de validation par étapes (sanity checks, ablations, et vérification des termes de log-prob *squash*).

8 Discussion : limites et pistes d’amélioration

8.1 Sensibilité aux seeds et variance inter-épisodes

La comparaison Run A vs Run C illustre que, même avec les mêmes hyperparamètres et le même budget (2M, $c_e = 0$), la seed peut conduire à des politiques finales très différentes en termes de robustesse. Cette variabilité est un point clé en RL : elle impose de raisonner en termes de distribution de performances (multi-runs) plutôt qu’en terme de meilleur run isolé.

8.2 Exploration via entropie

L’ajout d’entropie ($c_e = 0.005$) vise à augmenter l’exploration et potentiellement améliorer la robustesse (meilleure couverture de l’espace d’états). Néanmoins, un coefficient trop élevé peut ralentir la convergence ou maintenir une politique trop stochastique. Le bon réglage dépend typiquement de la dynamique observée dans les courbes (eval_avg_return, KL, clipfrac).

8.3 Pistes techniques (sans changer l’algorithme de base)

Dans la continuité PPO, plusieurs améliorations sont naturelles :

- **Target KL** : imposer un arrêt anticipé d’époque si la KL dépasse un seuil (stabilisation).
- **Multi-env** : vectoriser plusieurs environnements pour réduire la corrélation temporelle et accélérer la collecte.
- **Normalisation des avantages** : standardiser $\hat{A}_t$ (souvent bénéfique).
- **Ablations** : isoler l’impact de `value_clip`, du schedule LR, de c_e .

L’implémentation de cet algorithme nous a réellement intéressé, d’autant plus que nous sommes intéressés par de la recherche donc nous pensons continuer à explorer ces pistes à l’avenir.

9 Conclusion

Ce projet propose une implémentation from scratch structurée de PPO sur `BipedalWalker-v3`, intégrant les briques indispensables à un entraînement stable en action continue : politique gaussienne squashée, estimation GAE, objectif clipped, normalisation des observations, instrumentation (TensorBoard/diagnostics), checkpoints et évaluation périodique.

Les résultats montrent qu’il est possible d’atteindre des performances élevées (~ 313 – 314) et stables, mais que la robustesse dépend fortement des conditions expérimentales : une seed différente peut suffire à transformer un run instable (Run A) en un run robuste (Run C), et une augmentation de budget combinée à une exploration non nulle (Run B) peut stabiliser fortement l’apprentissage à seed fixe. Les prochaines étapes consistent à compléter la comparaison sur davantage de seeds et à systématiser l’analyse des comportements qualitatifs via les vidéos.

Annexe A — Extraits de configuration (pour reproductibilité)

Exemple de lancement (train.py) :

```
1  train(  
2      env_id="BipedalWalker-v3",  
3      total_steps=2_000_000,  
4      rollout_len=2048,  
5      gamma=0.99,  
6      lam=0.95,  
7      clip_eps=0.2,  
8      lr=3e-4,  
9      epochs=10,  
10     minibatch_size=64,  
11     vf_coef=0.5,  
12     ent_coef=0.0,  
13     max_grad_norm=0.5,  
14     target_kl=None,  
15     value_clip=True,  
16     seed=0,  
17     device="cuda" if torch.cuda.is_available() else  
18         "mps" if torch.backends.mps.is_available() else "cpu",  
19     save_video=True,  
20     video_every=25,  
21     save_ckpt_every=25,  
22 )
```